American Computer Science League

2023-2024 ● Contest 3: ACSL Rack-O ● Senior Division

PROBLEM: The game of Rack-O is played with each player being dealt 10 cards with numbers from 1 to 60 on them. Each player places those cards in the order dealt, from front to back, in a *rack* with slots for holding the cards. Through a series of draws from the draw *pile*, each player may replace a card in the rack with the drawn card. The goal is to have the cards in the rack in ascending order. The first player to do so is the winner.

In the ACSL version, you will be given the number of slots, *s*, and the number of cards, *n*, in the game. Each card has a unique number, between 1 and *n*, inclusive. You will be given *s* cards to place in the rack, from front to back, and then a set of cards forming the draw pile. You will pick up the next card from the draw pile and place it in the best possible slot in the rack or ignore it.

However, because you don't know what cards will be drawn later, the card cannot be placed in the perfect spot. We resort to using a *heuristic* approach - choosing a strategy value that is considered the best among alternatives.

The heuristic value we will use is the number of *step-downs*. A step-down occurs when a card is less than the card in the previous slot in the rack (e.g. 40, 20, 30, 50, 70 has 1 step-down after 40). When there are 0 step-downs, the entire rack is in ascending order which ends the game. Therefore, the lower the heuristic value, the better the rack.

Perform one of the following strategies if it is possible and it either decreases the heuristic value or keeps it the same. If the heuristic values are the same, choose Strategy A. Otherwise, ignore the drawn card.

A. **Ideal Slot:** Create an interval that is the ideal set of numbers for each slot. To determine the interval size for each slot, use the formula *n/s* if *n* is a multiple of *s* or int(*n/s*)+1 otherwise. For example, with 4 slots and 25 cards, the size of each interval is int(25/4)+1=7. If *n* is not a multiple of *s*, the last slot might have a smaller interval size. Determine the numbers that could possibly be placed in each slot. For example, the range of numbers are 1-7, 8-14, 15-21, and 22-25 for slots 1-4 respectively. Place the drawn card in the slot where it *ideally* belongs only if the card that is already in that slot is not in that interval. For example, if 20 is the drawn card, place it in slot 3 only if the card already there is not between 15 and 21, inclusive.

B. **Ascending Order:** Starting at the front of the rack, find the <u>first set of 3 cards</u>, not already in ascending order, where replacing one of those cards with the drawn card

puts the 3 cards in ascending order. For example, if the rack is 10 20 30 9 40, the first set of 3 adjacent numbers that is not already in order is 20 30 9. If the drawn card is 33 (or anything larger than 30), place it in the 4th slot. If the drawn card is 25, do not use it. Continue along the rack to find the next case if one exists.

As soon as the cards in the rack are in ascending order, the game ends and no more cards are drawn. The game also ends when the draw pile is empty. At the end of the game, output the entire rack as a string of numbers separated by a single space.

EXAMPLE:

Input:

```
9 70
40 35 30 56 32 58 44 17 45
31 37 10 28 21 62 7 64 16 12
```

Output:

```
7 10 21 31 32 37 44 62 64
```

Explanation:

The intervals for 9 slots and 70 cards are: 1-8, 9-16, 17-24, 25-32, 33-40, 41-48, 49-56, 57-64, 65-70. If the number 23 is drawn, its ideal slot is the 3rd slot, since the set of ideal numbers for slot 3 is 17-24. The following table shows how the game is played for each card drawn:

American Computer Science League

2023-2024 ● Contest 3: ACSL Rack-O ● Senior Division

Rack	Card Drawn	Heuristic value [slot]		Strategy used	Explanation
Starting rack: 40 35 30 56 32 58 44 17 45		**A**	**B**		The initial number of step-downs is 5.
40 35 30 56 32 58 44 17 45	31	4[4]	4[4]	A	Both improve the rack - use Strategy A.
40 35 30 31 32 58 44 17 45	37	4[5]	3[6]	B	Strategy B is a smaller value - use Strategy B.
40 35 30 31 32 37 44 17 45	10	2[2]	2[2]	A	Both improve the rack - use Strategy A.
40 10 30 31 32 37 44 17 45	28	x	x		Neither is possible - ignore the drawn card.
40 10 30 31 32 37 44 17 45	21	2[3]	x	A	The value stays the same - use Strategy A.
40 10 21 31 32 37 44 17 45	62	2[8]	2[8]	A	Both values stay the same - use Strategy A.
40 10 21 31 32 37 44 62 45	7	1[1]	1[1]	A	Both improve the rack - use Strategy A.
7 10 21 31 32 37 44 62 45	64	x	0[9]	B	Only B is possible - use it.
7 10 21 31 32 37 44 62 64	end				The value becomes 0 - the game ends.

INPUT: There will be three strings: the number of slots and number of cards, the cards in the rack, and the cards in the draw pile. Every number will be separated by a single space. There will be no more than 20 slots and no more than 200 possible cards.

OUTPUT: When the game ends, output the cards in the rack, from front to back, as a single string, with each number separated by a single space.

American Computer Science League

2023-2024 ● Contest 3: ACSL Rack-O ● Senior Division

SAMPLE INPUT:

```
1. 9 70
   40 35 30 56 32 58 44 17 45
   31 37 10 28 21 62 7 64 16 12
2. 15 90
   15 12 18 9 28 17 46 51 7 53 65 70 74 84 47
   45 73 3 52 54 16 21 44 87 40 68 30 37
3. 12 130
   20 110 30 16 84 40 91 69 75 7 81 15
   33 47 114 55 35 71 25 123 51 23 34 10 100 77 36 115
4. 8 100
   6 13 47 62 32 70 76 12
   3 67 80 10 39 44 2 43 90 85 21 63 4 52
5. 12 110
   44 35 22 25 79 100 85 69 87 3 56 28
   97 10 48 43 42 21 81 47 86 88 80 54 24 50
```

SAMPLE OUTPUT:

```
1. 7 10 21 31 32 37 44 62 64
2. 3 12 18 21 30 37 40 51 52 53 65 70 74 84 87
3. 10 25 30 34 35 40 47 69 75 100 114 123
4. 6 21 39 43 63 70 80 85
5. 10 35 81 86 88 24 42 80 87 97 56 28
```

American Computer Science League

2023-2024 ● Contest 3: ACSL Rack-O ● Senior Division

TEST DATA

TEST INPUT:

```
1. 9 140
   74 135 61 45 92 122 14 98 138
   105 60 66 116 5 106 97 85 18 139 96
2. 15 75
   27 43 24 9 70 64 3 33 30 63 11 1 25 12 35
   69 15 10 2 34 66 21 49 23 51 5 57 38 40 53
3. 11 85
   68 52 8 25 22 18 29 16 74 48 34
   30 43 6 77 7 28 73 9 27 54 71 79 42 49
4. 15 60
   2 10 13 19 20 26 30 34 39 41 47 48 52 58 60
   9 31 50 59 1
5. 10 75
   70 65 60 55 50 45 40 35 30 20
   44 10 15 46 23 72 3 68 53 37 52 75 39 51 18 71 73 62
```

TEST OUTPUT:

```
1. 5 18 61 66 92 97 106 116 138
2. 2 43 53 9 23 64 66 10 30 49 51 57 25 34 35
3. 6 7 8 25 30 43 54 71 74 77 79
4. 2 10 13 19 20 26 30 34 39 41 47 48 52 58 60
5. 3 10 23 46 50 51 52 53 72 75
```

American Computer Science League

2023-2024 ● Contest 3: ACSL Rack-O ● Senior Division

问题：Rack-O 游戏中，每位玩家都会收到 10 张卡牌，每张卡牌上都印有一个位于1 到 60 之间的数字。每位玩家按照发牌顺序，将这 10张卡牌从前至后摆放在一个带卡槽的*架子*上。每位玩家从抽牌*堆*中抽牌，在一系列抽取的过程中，玩家可以用被抽到的卡牌替换在牌架上的一张卡牌。游戏目标是使得牌架上所有卡牌按升序排列。第一位达成游戏目标的玩家获胜。

在 ACSL 版的 Rack-O 游戏中，玩家会被告知卡槽数量 s，以及卡牌数量 n。每张卡牌上都有一个在1到 n 之间（包括 1 和 n）的专属数字。玩家将会收到s 张卡牌，然后将这些卡牌从前至后摆放在牌架上。除此之外，玩家还会收到一叠卡牌，形成一个抽牌堆。玩家从抽牌堆中抽取下一张卡牌，然后将其放在牌架上尽可能最佳的位置（卡槽）或者放弃这张卡牌。

但是，由于玩家不知道之后会抽到什么卡牌，所以这张卡牌无法被放在最佳放置上。对此，我们使用 *启发式* 方法——在各种可选策略中选择被认为是最佳的策略值。

我们将采用的启发式值指的是 *递减* 次数。在此游戏中，递减指的是牌架上某一张卡牌小于其前一卡槽中的卡牌（例如，在 40、20、30、50、70中，数字 40 后发生一次递减）。如果牌架上有 0 次递减，则表示整个牌架是按照升序排列，此时游戏结束。因此，启发式值越低，牌架上的卡牌排列情况越接近游戏目标。

如果以下策略中的某一种策略**可行**的话，则执行这种策略。这种策略要么能降低启发式值，要么保持启发式值不变。如果启发式值不变，则选择策略 A。否则，放弃抽到的卡牌。

A. **理想卡槽：**为每个卡槽创建一个区间，这个区间是一个理想的数字组合。为了确定每个卡槽的区间大小，可以使用公式。如果 n 是 s 的倍数，则使用公式 n/s；否则使用公式 int(n/s)+1。例如，如果有 4 个卡槽和 25 张牌，则每个区间的大小为 int(25/4)+1=7。如果 n 不是 s 的倍数，则最后一个卡槽的区间可能较小。接下来，确定每个卡槽中可能可以放置的数字。例如，对于卡槽 1 - 4，每个卡槽可放置的数字范围分别为 1-7、8-14、15-21、22-25。被抽到的卡牌可以被放在它理想的卡槽的前提是该卡槽中已经存在的卡牌不在其理想区间内。例如，如果抽到的卡牌 20，则只有当卡槽 3 中已经存在的卡牌不在 15 到 21 之间（包括 15 和 21）时，才将其放在其卡槽 3 中。

B. **升序：**从牌架前方开始，找到<u>第一组未按升序排列的3张卡牌</u>。在这一组卡牌中，用抽到的卡牌替换其中某一张卡牌使得这 3张卡牌按升序排列。例如，如果牌架上

是 10、20、30、9、40，那么第一组 3 个相邻但未按升序排列的卡牌数字就是 20 、30、9。如果抽到的卡牌是 33（或者任何一个比 30 大的数），则将其放入第 4 个卡槽中。如果抽到的卡牌是 25，则放弃使用这张卡牌。如果还存在其他类似的组合，则继续沿着牌架寻找下一组卡牌。

一旦牌架上的卡牌按升序排列，游戏结束，不再抽牌。当抽牌堆为空时，游戏也会结束。游戏结束时，以一个数字字符串的形式输出整个牌架，各数字之间用一个空格隔开。

样 例 ：

输入：

```
9 70
40 35 30 56 32 58 44 17 45
31 37 10 28 21 62 7 64 16 12
```

输出：

```
7 10 21 31 32 37 44 62 64
```

解析：

9 个卡槽和 70 张卡牌的区间为：1-8、9-16、17-24、25-32、33-40、41-48、49-56、57-64、65 -70。如果抽到 23，那么其理想卡槽为第 3 个卡槽，因为卡槽 3 的理想数字组合是17-24。每次抽牌时的游戏操作详见下表：

American Computer Science League

2023-2024 ● Contest 3: ACSL Rack-O ● Senior Division

牌架	抽牌	启发式值 [卡槽]		使用策略	解析
起始牌架： 40 35 30 56 32 58 44 17 45		A	B		初始递减次数为 5 次。
40 35 30 56 32 58 44 17 45	31	4[4]	4[4]	A	两者都能优化牌架 -使用策略A。
40 35 30 31 32 58 44 17 45	37	4[5]	3[6]	B	策略 B 的启发式值更小-使用策略 B。
40 35 30 31 32 37 44 17 45	10	2[2]	2[2]	A	两者都能优化牌架 -使用策略A。
40 10 30 31 32 37 44 17 45	28	x	x		两者都不可行-放弃抽到的牌
40 10 30 31 32 37 44 17 45	21	2[3]	x	A	启发式值保持不变 -使用策略 A。
40 10 21 31 32 37 44 17 45	62	2[8]	2[8]	A	两者都不改变启发式值-使用策略 A。
40 10 21 31 32 37 44 62 45	7	1[1]	1[1]	A	两者都能优化牌架 -使用策略A。
7 10 21 31 32 37 44 62 45	64	x	0[9]	B	只有策略 B 可行-使用策略 B。
7 10 21 31 32 37 44 62 64	结束				启发式值变为 0 -游戏结束。

输入：输入三个字符串：卡槽数量及卡牌数量；牌架上的卡牌；抽牌堆中的卡牌。各个数字之间用一个空格隔开。卡槽数量不超过 20 个，可能有的卡牌数量不超过 200 个

输出：游戏结束时，以一个字符串的形式从前至后输出牌架上的卡牌，其中每个数字之间用一个空格隔开。

样例输入：

```
1. 9 70
   40 35 30 56 32 58 44 17 45
   31 37 10 28 21 62 7 64 16 12
2. 15 90
   15 12 18 9 28 17 46 51 7 53 65 70 74 84 47
   45 73 3 52 54 16 21 44 87 40 68 30 37
3. 12 130
   20 110 30 16 84 40 91 69 75 7 81 15
   33 47 114 55 35 71 25 123 51 23 34 10 100 77 36 115
4. 8 100
   6 13 47 62 32 70 76 12
   3 67 80 10 39 44 2 43 90 85 21 63 4 52
5. 12 110
   44 35 22 25 79 100 85 69 87 3 56 28
   97 10 48 43 42 21 81 47 86 88 80 54 24 50
```

样例输出：

```
1. 7 10 21 31 32 37 44 62 64
2. 3 12 18 21 30 37 40 51 52 53 65 70 74 84 87
3. 10 25 30 34 35 40 47 69 75 100 114 123
4. 6 21 39 43 63 70 80 85
5. 10 35 81 86 88 24 42 80 87 97 56 28
```

American Computer Science League

2023-2024 ● Contest 3: ACSL Rack-O ● Senior Division

测试数据

测试输入：

```
1. 9 140
   74 135 61 45 92 122 14 98 138
   105 60 66 116 5 106 97 85 18 139 96
2. 15 75
   27 43 24 9 70 64 3 33 30 63 11 1 25 12 35
   69 15 10 2 34 66 21 49 23 51 5 57 38 40 53
3. 11 85
   68 52 8 25 22 18 29 16 74 48 34
   30 43 6 77 7 28 73 9 27 54 71 79 42 49
4. 15 60
   2 10 13 19 20 26 30 34 39 41 47 48 52 58 60
   9 31 50 59 1
5. 10 75
   70 65 60 55 50 45 40 35 30 20
   44 10 15 46 23 72 3 68 53 37 52 75 39 51 18 71 73 62
```

测试输出：

```
1. 5 18 61 66 92 97 106 116 138
2. 2 43 53 9 23 64 66 10 30 49 51 57 25 34 35
3. 6 7 8 25 30 43 54 71 74 77 79
4. 2 10 13 19 20 26 30 34 39 41 47 48 52 58 60
5. 3 10 23 46 50 51 52 53 72 75
```